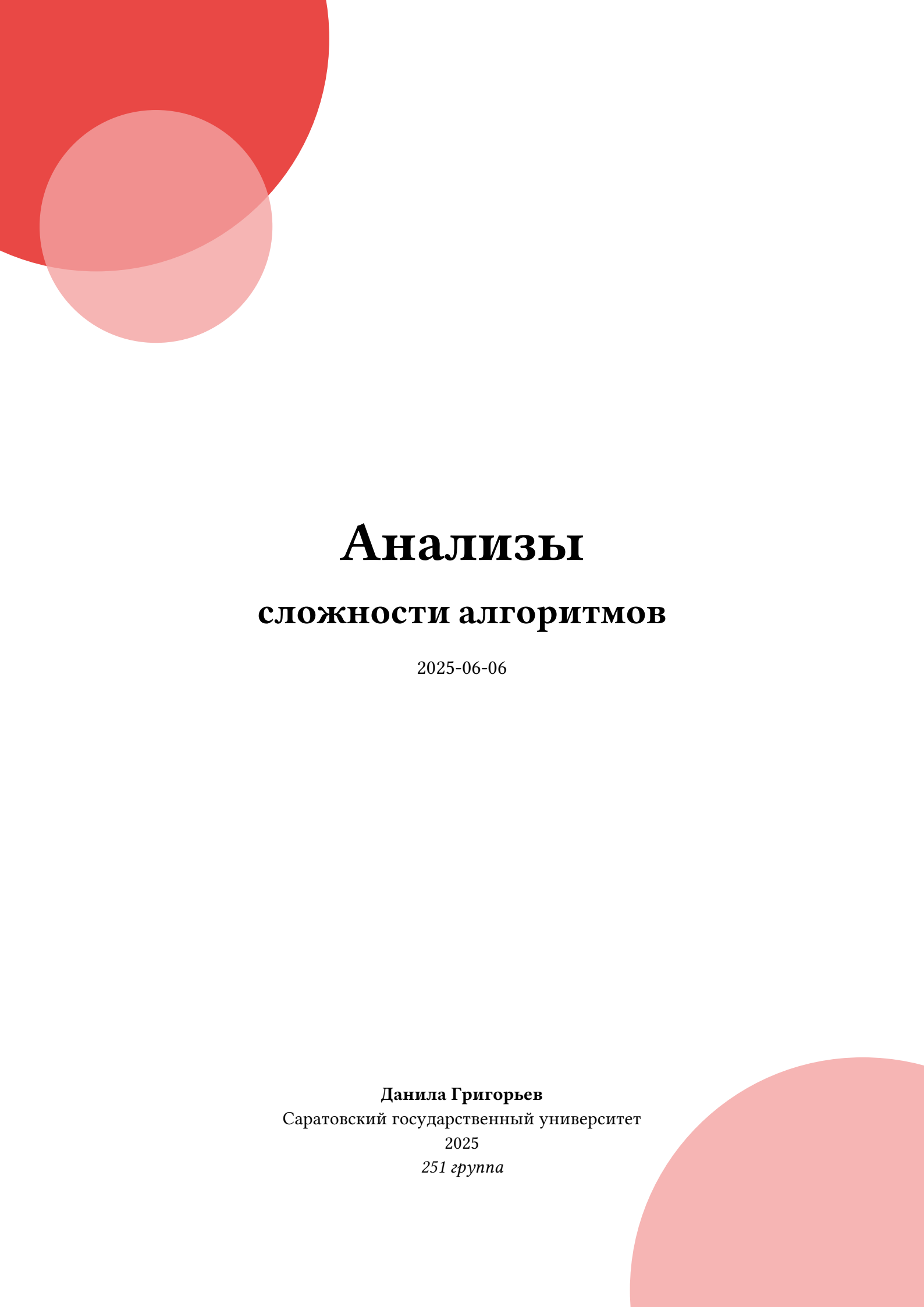




Анализы

сложности алгоритмов

2025-06-06

Данила Григорьев
Саратовский государственный университет
2025
251 группа

I. Анализ сложности строк, не требующих сравнение элементов

I.1. Сортировка подсчётом

Реализация

```
namespace Sortings {
    void counting_sort(std::vector<int, std::allocator<int>>& arr) {
        if (arr.empty()) return;

        int min_val = *std::min_element(arr.begin(), arr.end());
        int max_val = *std::max_element(arr.begin(), arr.end());
        int range = max_val - min_val + 1;

        if (range > 1000000) {
            std::cerr << "Error: Range too large for counting sort.\n";
            return;
        }

        std::vector<int, std::allocator<int>> count(range, 0);
        std::vector<int, std::allocator<int>> output(arr.size());

        for (int num : arr) count[num - min_val]++;
        for (int i = 1; i < range; ++i) count[i] += count[i - 1];
        for (int i = arr.size() - 1; i >= 0; --i) {
            int pos = count[arr[i] - min_val] - 1;
            output[pos] = arr[i];
            count[arr[i] - min_val]--;
        }

        arr = output;
    }
}
```

Анализ сложности

Действие	Сложность
Проверка на пустой массив	O(1)
Определение разброса значений (поиск минимума и максимума, вычисление разности)	O(n)
Аллокация вспомогательных векторов, в том числе вектора размерностью k (range) для подсчёта	O(1)
Подсчёт вхождений каждого элемента	O(n)
Заполнение нового отсортированного вектора	O(n + k)
Копирование значений во входной вектор	O(n)

Общая временная сложность:

$O(n + k)$, n — количество элементов в исходном массиве, k — это разница между минимумом и максимумом

1.2. Поразрядная сортировка

Реализация

```
namespace Sortings {
    void counting_sort_by_digit(std::vector<int>& arr, int exp) {
        std::vector<int> output(arr.size());
        std::vector<int> count(10, 0);

        for (int num : arr) count[(num / exp) % 10]++;
        for (int i = 1; i < 10; ++i) count[i] += count[i - 1];
        for (int i = arr.size() - 1; i >= 0; --i) {
            int digit = (arr[i] / exp) % 10;
            output[count[digit] - 1] = arr[i];
            count[digit]--;
        }

        arr = output;
    }

    void radix_sort_lsd_positive(std::vector<int>& arr) {
        if (arr.empty()) return;

        int max_num = *std::max_element(arr.begin(), arr.end());
        for (int exp = 1; max_num / exp > 0; exp *= 10)
            counting_sort_by_digit(arr, exp);
    }

    void radix_sort_lsd(std::vector<int>& arr) {
        std::vector<int> neg, pos;
        for (int num : arr) {
            if (num < 0) neg.push_back(-num);
            else pos.push_back(num);
        }

        radix_sort_lsd_positive(neg);
        std::reverse(neg.begin(), neg.end());
        for (int& num : neg) num = -num;

        radix_sort_lsd_positive(pos);

        arr.clear();
        arr.insert(arr.end(), neg.begin(), neg.end());
        arr.insert(arr.end(), pos.begin(), pos.end());
    }
}
```

Анализ сложности

Действие	Сложность
Проверка на пустой массив	$O(1)$
Разделение массива на положительные и отрицательные	$O(n)$
Сортировка положительных чисел в количестве m штук с максимальным числом разрядов k	$O(m \cdot k)$
Сортировка отрицательных чисел по аналогии	$O((n - m) \cdot g)$

Итоговая сложность — $O(m \cdot k + (n - m) \cdot g)$

II. Анализ быстрой и пирамидальной сортировок**II.1. Быстрая сортировка****Реализация**

```
namespace Sortings {
    void quick_sort(std::vector<int>& arr, bool debug) {
        std::random_device rdev;
        std::default_random_engine rengine(rdev());
        if(arr.size() <= 1) {
            return;
        }
        std::function<void(int, int)> partition_sort;
        partition_sort = [&arr, &rengine, &partition_sort, debug](int low, int
high) {
            if(low >= high) {
                if(debug) {
                    std::cout << " low >= high." << std::endl << std::endl;
                }
                return;
            }
            std::uniform_int_distribution<> unidist(0, high - low);
            int pivot_index = low + unidist(rengine);
            int pivot = arr[pivot_index];
            std::swap(arr[pivot_index], arr[high]);
            int i = low - 1;
            for(int j = low; j < high; ++j) {
                if(arr[j] <= pivot) {
                    i++;
                    std::swap(arr[i], arr[j]);
                }
            }
            std::swap(arr[i + 1], arr[high]);
            int pivot_pos = i + 1;
            partition_sort(low, pivot_pos - 1);
            partition_sort(pivot_pos + 1, high);
        };
        partition_sort(0, arr.size() - 1);
    }
}
```

```

    }
}

```

Анализ сложности

Для цикла временная сложность равна $O(n)$, где n — длина отрезка

Если удачно выбран опорный элемент, то в лучшем случае алгоритм пройдет по всем n элементам в массиве и поделит его рекурсивно пополам $\log n$ раз

Согласно теореме о рекурсивном вызове:

$T(n) = c$, если $n = 1$; $T(\frac{n}{k}) + bn^a$, если $n > 1$

$$T(n) = \begin{cases} c \\ 2T(\frac{n}{2}) + bn \end{cases}$$

$a = 2$ — количество порождаемых рекурсией подзадач, $\frac{n}{k}$ — размер подзадач, $k = 2$ — постоянная.

Трудоёмкость рекурсивного перехода имеет сложность $O(n)$

По следствию теоремы для лучшего случая $T(n) = 2T(\frac{n}{2}) + O(n) \Rightarrow T(n) = O(n \log n)$

При неудачном выборе опорного элемента (например, самый большой или самый маленький элемент), массив делится на части размера $n - 1$ и 1 . Таким образом количество уровней рекурсии будет n . На каждом уровне рекурсии мы выполняем $O(n)$ операций по разделению массива и сравнению элементов. В сумме на всех уровнях это даёт $O(n^2)$.

Для худшего случая получим $T(n) = T(1) + T(n - 1) = T(n) = O(n^2)$

II.2. Пирамидальная сортировка

Реализация

```

namespace Sortings {
    void heapify_down(std::vector<int>& arr, size_t n, size_t i) {
        size_t largest = i;
        size_t left = 2 * i + 1;
        size_t right = 2 * i + 2;

        if (left < n && arr[left] > arr[largest]) {
            largest = left;
        }
        if (right < n && arr[right] > arr[largest]) {
            largest = right;
        }
        if (largest != i) {
            std::swap(arr[i], arr[largest]);
            heapify_down(arr, n, largest);
        }
    }
}

```

```

void make_heap(std::vector<int>& arr) {
    size_t n = arr.size();
    for (int i = (n / 2) - 1; i >= 0; --i) {
        heapify_down(arr, n, i);
    }
}

// Удаляет самый большой (т. е. корневой) элемент кучи
void pop_heap(std::vector<int>& arr, size_t n) {
    if (n <= 1) return;
    std::swap(arr[0], arr[n - 1]);
    heapify_down(arr, n - 1, 0);
}

void heap_sort(std::vector<int>& arr, bool debug) {
    if (arr.size() <= 1) return;
    make_heap(arr.begin(), arr.end());
    if(debug) {
        std::cout << ' ';
        Utils::print_array(arr);
    }
    for (size_t i = arr.size() - 1; i > 0; --i) {
        pop_heap(arr.begin(), arr.begin() + i + 1);
        if(debug) {
            std::cout << " i = " << i << ":\n ";
            Utils::print_array(arr);
        }
    }
}
}

```

Анализ сложности

Построение кучи: Общая сложность построения кучи – $O(n)$. Результат основывается на сумме высот всех узлов.

Сортировка (извлечение элементов): Цикл выполняется $n-1$ раз, каждый раз вызывается функция извлечения с временной сложностью $O(\log n)$.

Общая временная сложность: $O(n \cdot \log n)$.

III. Двоичное дерево

Реализация

```

#pragma once
#include <cmath>
#include <bits/stdc++.h>
#include <iostream>
#include <queue>
#include <vector>

template <typename T>

```

```

class Node {
public:
    T key;
    Node* left;
    Node* right;
    Node(T k);
};

template <typename T>
class BinaryTree {
public:
    BinaryTree();
    ~BinaryTree();

    void add_node(T key);
    void delete_node(T key);
    Node<T>* get_root();
    Node<T>* find(T key);
    std::vector<T> in_order();
    std::vector<T> pre_order();
    std::vector<T> post_order();
    void print_tree_vertical(Node<T>* node, int space = 0, int indent = 4);
    void print() const;

private:
    Node<T>* root;

    void destroyTree(Node<T>* node);
    void add_node_recursive(Node<T>* node, T key);
    Node<T>* delete_node_recursive(Node<T>* node, T key);
    Node<T>* find_min(Node<T>* node);
    Node<T>* find_recursive(Node<T>* node, T key);
    void in_order_recursive(Node<T>* node, std::vector<T>& result);
    void pre_order_recursive(Node<T>* node, std::vector<T>& result);
    void post_order_recursive(Node<T>* node, std::vector<T>& result);
    int get_height(Node<T>* node) const;
    void print_helper(std::vector<std::vector<Node<T>*>>& arr, Node<T>* x, int
deepness, int ind) const;
    std::string format_node(Node<T>* node) const;
};

template <typename T>
void BinaryTree<T>::print() const {
    if (!root) return;

    int max_height = get_height(root);
    int offset = 1;
    int width = 1;
    for (int i = 1; i < max_height; ++i) offset <=< 1;

    // Создаем двумерный массив указателей на узлы
    std::vector<std::vector<Node<T>*>> arr(max_height);

```

```

    for (int i = 0, w = 1; i < max_height; ++i, w <= 1)
        arr[i].resize(w, nullptr);

    print_helper(arr, root, 0, 0);

    int curr_offset = offset;
    int curr_width = 1;
    for (int i = 0; i < max_height; ++i) {
        std::cout << std::setw((curr_offset >> 1) + 1);
        std::cout << format_node(arr[i][0]);
        for (int j = 1; j < curr_width; ++j) {
            std::cout << std::setw(curr_offset);
            std::cout << format_node(arr[i][j]);
        }
        curr_offset >= 1;
        curr_width <= 1;
        std::cout << std::endl;
    }
}

// Вспомогательные методы для печати
template <typename T>
int BinaryTree<T>::get_height(Node<T>* node) const {
    if (!node) return 0;
    return 1 + std::max(get_height(node->left), get_height(node->right));
}

template <typename T>
void BinaryTree<T>::print_helper(std::vector<std::vector<Node<T>*>>& arr,
Node<T>* x, int deepness, int ind) const {
    arr[deepness][ind] = x;
    if (x && x->left) {
        print_helper(arr, x->left, deepness + 1, 2 * ind);
    }
    if (x && x->right) {
        print_helper(arr, x->right, deepness + 1, 2 * ind + 1);
    }
}

template <typename T>
std::string BinaryTree<T>::format_node(Node<T>* node) const {
    if (!node) return " ";
    return std::to_string(node->key);
}

template <typename T>
void BinaryTree<T>::print_tree_vertical(Node<T>* node, int space, int indent)
{
    if (node == nullptr) return;

    // Сначала печатаем правое поддерево
    print_tree_vertical(node->right, space + indent, indent);

```



```

// Печатаем текущий узел с отступом
std::cout << std::setw(space) << "" << node->key << std::endl;

// Затем печатаем левое поддерево
print_tree_vertical(node->left, space + indent, indent);
/*if (!root) {
    std::cout << "<empty tree>\n";
    return;
}

// Узнаём высоту дерева
std::function<int(Node<T>*)> tree_height = [&](Node<T>* node) {
    if (!node) return 0;
    return 1 + std::max(tree_height(node->left), tree_height(node-
>right));
};
int height = tree_height(root);

// Используем очередь для обхода по уровням
std::queue<Node<T>*> q;
q.push(root);

int max_width = std::pow(2, height) * 4; // ширина для форматирования

for (int level = 0; level < height; ++level) {
    int level_nodes = std::pow(2, level);
    int space_between = max_width / level_nodes;
    int first_space = space_between / 2;

    // Выводим узлы текущего уровня
    for (int i = 0; i < level_nodes; ++i) {
        Node<T>* node = q.front();
        q.pop();

        if (i == 0)
            std::cout << std::setw(first_space) << "";

        if (node) {
            std::cout << node->key;
            q.push(node->left);
            q.push(node->right);
        } else {
            std::cout << " ";
            q.push(nullptr);
            q.push(nullptr);
        }
        std::cout << std::setw(space_between) << "";
    }
    std::cout << "\n";
}*/
}

```

```
template <typename T>
Node<T>::Node(T k) : key(k), left(nullptr), right(nullptr) {}

template <typename T>
BinaryTree<T>::BinaryTree() : root(nullptr) {}

template <typename T>
BinaryTree<T>::~~BinaryTree() {
    destroyTree(root);
}

template <typename T>
void BinaryTree<T>::add_node(T key) {
    if (!root) {
        root = new Node<T>(key);
    } else {
        add_node_recursive(root, key);
    }
}

template <typename T>
void BinaryTree<T>::destroyTree(Node<T>* node) {
    if (node) {
        destroyTree(node->left);
        destroyTree(node->right);
        delete node;
    }
}

template <typename T>
void BinaryTree<T>::add_node_recursive(Node<T>* node, T key) {
    if (key < node->key) {
        if (!node->left) {
            node->left = new Node<T>(key);
        } else {
            add_node_recursive(node->left, key);
        }
    } else if (key > node->key) {
        if (!node->right) {
            node->right = new Node<T>(key);
        } else {
            add_node_recursive(node->right, key);
        }
    }
}

template <typename T>
Node<T>* BinaryTree<T>::delete_node_recursive(Node<T>* node, T key) {
    if (!node) return nullptr;

    if (key < node->key) {
```

```

    node->left = delete_node_recursive(node->left, key);
} else if (key > node->key) {
    node->right = delete_node_recursive(node->right, key);
} else {
    if (!node->left) {
        Node<T>* temp = node->right;
        delete node;
        return temp;
    } else if (!node->right) {
        Node<T>* temp = node->left;
        delete node;
        return temp;
    }
    Node<T>* min_node = find_min(node->right);
    node->key = min_node->key;
    node->right = delete_node_recursive(node->right, min_node->key);
}
return node;
}

template <typename T>
Node<T>* BinaryTree<T>::find_min(Node<T>* node) {
    while (node->left) {
        node = node->left;
    }
    return node;
}

template <typename T>
void BinaryTree<T>::delete_node(T key) {
    root = delete_node_recursive(root, key);
}

template <typename T>
Node<T>* BinaryTree<T>::find(T key) {
    return find_recursive(root, key);
}

template <typename T>
Node<T>* BinaryTree<T>::find_recursive(Node<T>* node, T key) {
    if (!node || node->key == key) return node;
    if (key < node->key) return find_recursive(node->left, key);
    return find_recursive(node->right, key);
}

template <typename T>
Node<T>* BinaryTree<T>::get_root() {
    return root;
}

template <typename T>
std::vector<T> BinaryTree<T>::in_order() {

```

```

    std::vector<T> result;
    in_order_recursive(root, result);
    return result;
}

template <typename T>
void BinaryTree<T>::in_order_recursive(Node<T>* node, std::vector<T>& result)
{
    if (node) {
        in_order_recursive(node->left, result);
        result.push_back(node->key);
        in_order_recursive(node->right, result);
    }
}

template <typename T>
std::vector<T> BinaryTree<T>::pre_order() {
    std::vector<T> result;
    pre_order_recursive(root, result);
    return result;
}

template <typename T>
void BinaryTree<T>::pre_order_recursive(Node<T>* node, std::vector<T>& result)
{
    if (node) {
        result.push_back(node->key);
        pre_order_recursive(node->left, result);
        pre_order_recursive(node->right, result);
    }
}

template <typename T>
std::vector<T> BinaryTree<T>::post_order() {
    std::vector<T> result;
    post_order_recursive(root, result);
    return result;
}

template <typename T>
void BinaryTree<T>::post_order_recursive(Node<T>* node, std::vector<T>&
result) {
    if (node) {
        post_order_recursive(node->left, result);
        post_order_recursive(node->right, result);
        result.push_back(node->key);
    }
}

```

Анализ

n — количество элементов в дереве, h — высота дерева

Виды бинарного дерева

Худший случай. Дерево является вырожденным. У каждого узла только правый или только левый потомок. Высота дерева в этом случае будет равна $= n$

Лучший случай. Дерево сбалансировано. Высота одного поддерева отличается от высоты другого не более чем на 1. Высота такого дерева равна $\log n$

Средний случай. Любое дерево, не являющееся вырожденным и сбалансированным. Высота(h): $\log n < h < n$

Вставка элемента

Максимальное количество сравнений зависит от высоты дерева.

Худший случай. В худшем случае вставка произойдет за $O(n)$ сравнений, в крайний правый или левый узел.

Лучший случай. В лучшем случае вставка произойдет за $O(\log n)$ сравнений.

Средний случай. В среднем случае вставка произойдет за $O(\log n)$ сравнений.

Поиск элемента

Лучший случай. Проходим через узлы один за другим. Если мы найдем элемент на втором уровне, то мы сделаем 2 сравнения, если на третьем, то 3 и т. д. Таким образом, на поиск ключа мы затратим время равное высоте дерева, то есть $\log n$, поэтому временная сложность поиска в лучшем случае составит $O(\log n)$

Худший случай. В худшем случае будет поиск крайнего правого или левого узла, за $O(n)$ сравнений

Средний случай. Высота дерева равна $\log n$. В среднем будет проведено $O(\log n)$ сравнений, как и в лучшем случае.

Обходы

Все обходы занимают $O(n)$, так как нужно пройти по всем элементам.

Удаление

- Если в дереве только корень, то нужно перезаписать его на null $O(1)$
- Если нет детей, то мы заменяем указатели родителей удаляемого элемента на null $O(1)$
- Если только один ребенок, то переобозначаем связи и удаление займет $O(1)$
- Если есть оба ребенка, то находим следующий. Это выполняется за $O(\log n)$. После переобозначаем связи и удаление объекта. Конечная сложность $O(1) + O(\log n) = O(\log n)$.

В коде имеется фиксированное количество операций сравнения. Удаление занимает $O(1)$, без учета на вызов функции поиска для нахождения узла, который нужно удалить

IV. Красно-чёрное дерево

IV.1. Реализация

```
#pragma once
#include <iomanip>
#include <iostream>

enum Color { BLACK, RED };

struct RedBlackTree {
public:
    // Node structure
    struct Node {
        Color color;
        int value;
        Node *right;
        Node *left;
        Node *parent;

        Node(int val) : color(RED), value(val), right(nullptr), left(nullptr),
parent(nullptr) {}
    };
    // Constructors
    RedBlackTree() : root(nullptr) {}

    // Public methods
    void insert(int value);
    void remove(int value);
    Node* find(int value);
    void print();

private:
    // Root pointer
    Node* root;

    // Helper methods
    Node* createNode(int value);
    void leftTurn(Node*& root, Node* node);
    void rightTurn(Node*& root, Node* node);
    Node* findGrandParent(Node* node);
    Node* findUncle(Node* node);
    Node* findSibling(Node* node);
    void insertCase1(Node*& root, Node* node);
    void insertCase2(Node*& root, Node* node);
    void insertCase3(Node*& root, Node* node);
    void insertCase4(Node*& root, Node* node);
    void insertCase5(Node*& root, Node* node);
    void deleteCase1(Node*& root, Node* node);
    void deleteCase2(Node*& root, Node* node);
    void deleteCase3(Node*& root, Node* node);
```

```

    void deleteCase4(Node*& root, Node* node);
    void deleteCase5(Node*& root, Node* node);
    void deleteCase6(Node*& root, Node* node);
    void replaceNode(Node*& root, Node* node);
    void removeOne(Node*& root, Node* node);
    void printHelper(Node*** nodes, Node* current, size_t depth = 0, size_t
index = 0);
    void max_height(Node* root, size_t& max, size_t depth = 1);
    std::string formatNode(Node* node);
};

#include "rb_tree.h"
#include <iomanip>
#include <iostream>
#include <cmath>

using namespace std;

// Create a new node
RedBlackTree::Node* RedBlackTree::createNode(int value) {
    return new Node(value);
}

// Left rotation
void RedBlackTree::leftTurn(Node*& root, Node* node) {
    Node* rightChild = node->right;
    node->right = rightChild->left;

    if (rightChild->left) {
        rightChild->left->parent = node;
    }
    rightChild->parent = node->parent;

    if (node->parent) {
        if (node == node->parent->left) {
            node->parent->left = rightChild;
        } else {
            node->parent->right = rightChild;
        }
    }

    rightChild->left = node;
    node->parent = rightChild;

    if (!rightChild->parent) {
        root = rightChild;
        root->color = BLACK;
    }
}

// Right rotation
void RedBlackTree::rightTurn(Node*& root, Node* node) {

```

```

Node* leftChild = node->left;
node->left = leftChild->right;

if (leftChild->right) {
    leftChild->right->parent = node;
}
leftChild->parent = node->parent;

if (node->parent) {
    if (node == node->parent->left) {
        node->parent->left = leftChild;
    } else {
        node->parent->right = leftChild;
    }
}

leftChild->right = node;
node->parent = leftChild;

if (!leftChild->parent) {
    root = leftChild;
    root->color = BLACK;
}
}

// Find grandparent
RedBlackTree::Node* RedBlackTree::findGrandParent(Node* node) {
    return (node && node->parent) ? node->parent->parent : nullptr;
}

// Find uncle
RedBlackTree::Node* RedBlackTree::findUncle(Node* node) {
    Node* grandParent = findGrandParent(node);
    if (!grandParent)
        return nullptr;
    return (grandParent->left == node->parent) ? grandParent->right
                                                : grandParent->left;
}

// Find sibling
RedBlackTree::Node* RedBlackTree::findSibling(Node* node) {
    if (!node || !node->parent)
        return nullptr;
    return (node->parent->left == node) ? node->parent->right
                                        : node->parent->left;
}

// Insertion cases
void RedBlackTree::insertCase1(Node*& root, Node* node) {
    if (!node->parent) {
        node->color = BLACK;
    } else {

```



```

        insertCase2(root, node);
    }
}

void RedBlackTree::insertCase2(Node*& root, Node* node) {
    if (node->parent->color == BLACK)
        return;
    insertCase3(root, node);
}

void RedBlackTree::insertCase3(Node*& root, Node* node) {
    Node* uncle = findUncle(node);
    Node* grandParent = findGrandParent(node);

    if (uncle && uncle->color == RED && node->parent->color == RED) {
        uncle->color = BLACK;
        node->parent->color = BLACK;
        grandParent->color = RED;
        insertCase1(root, grandParent);
    } else {
        insertCase4(root, node);
    }
}

void RedBlackTree::insertCase4(Node*& root, Node* node) {
    Node* grandParent = findGrandParent(node);

    if (node == node->parent->right && grandParent->left == node->parent) {
        leftTurn(root, node->parent);
        node = node->left;
    } else if (node == node->parent->left && grandParent->right == node->parent) {
        rightTurn(root, node->parent);
        node = node->right;
    }
    insertCase5(root, node);
}

void RedBlackTree::insertCase5(Node*& root, Node* node) {
    Node* grandParent = findGrandParent(node);
    grandParent->color = RED;
    node->parent->color = BLACK;

    if (node == node->parent->left && grandParent->left == node->parent) {
        rightTurn(root, grandParent);
    } else {
        leftTurn(root, grandParent);
    }
}

// Public methods
void RedBlackTree::insert(int value) {

```

```

    if (!root) {
        root = createNode(value);
        root->color = BLACK;
        return;
    }

    Node* current = root;
    while (true) {
        if (value < current->value) {
            if (!current->left) {
                current->left = createNode(value);
                current->left->parent = current;
                insertCase1(root, current->left);
                return;
            }
            current = current->left;
        } else if (value > current->value) {
            if (!current->right) {
                current->right = createNode(value);
                current->right->parent = current;
                insertCase1(root, current->right);
                return;
            }
            current = current->right;
        } else {
            return; // Value already exists
        }
    }
}

void RedBlackTree::remove(int value) {
    if (!root) return;

    Node* current = root;
    while (current && current->value != value) {
        if (value < current->value) {
            current = current->left;
        } else {
            current = current->right;
        }
    }

    if (current) {
        removeOne(root, current);
    }
}

RedBlackTree::Node* RedBlackTree::find(int value) {
    Node* current = root;
    while (current) {
        if (value < current->value) {
            current = current->left;
        }
    }
}

```

```

        } else if (value > current->value) {
            current = current->right;
        } else {
            return current;
        }
    }
    return nullptr;
}

void RedBlackTree::print() {
    std::cout << std::endl;
    if (!root) {
        std::cout << "Пустое дерево" << std::endl;
        return;
    }

    size_t maxDepth = 0;
    max_height(root, maxDepth);
    const short width = 1 << (maxDepth - 1);

    Node ***nodes = new Node **[maxDepth];
    for (size_t i = 0; i < maxDepth; i++) {
        nodes[i] = new Node*[width];
        fill_n(nodes[i], width, nullptr);
    }

    printHelper(nodes, root, 0, 0);
    for (size_t i = 0; i < maxDepth; i++) {
        short spacing = (1 << (maxDepth - i - 1)) * 4;
        for (size_t j = 0; j < (1 << i); j++) {
            cout << setw(j == 0 ? spacing / 2 : spacing) <<
formatNode(nodes[i][j]);
        }
        std::cout << std::endl;
    }

    for (size_t i = 0; i < maxDepth; i++) {
        delete[] nodes[i];
    }
    delete[] nodes;
    std::cout << std::endl;
}

// Helper methods for deletion
void RedBlackTree::deleteCase1(Node*& root, Node* node) {
    if (!node->parent) {
        root = (node->left) ? node->left : node->right;
        if (root)
            root->color = BLACK;
    } else {
        deleteCase2(root, node);
    }
}

```

```

}

void RedBlackTree::deleteCase2(Node*& root, Node* node) {
    Node* sibling = findSibling(node);

    if (sibling && sibling->color == RED) {
        node->parent->color = RED;
        sibling->color = BLACK;

        if (node == node->parent->left) {
            leftTurn(root, node->parent);
        } else {
            rightTurn(root, node->parent);
        }
    }
    deleteCase3(root, node);
}

void RedBlackTree::deleteCase3(Node*& root, Node* node) {
    Node* sibling = findSibling(node);

    if (sibling && node->parent->color == BLACK && sibling->color == BLACK &&
        (!sibling->left || sibling->left->color == BLACK) &&
        (!sibling->right || sibling->right->color == BLACK)) {
        sibling->color = RED;
        deleteCase1(root, node->parent);
    } else {
        deleteCase4(root, node);
    }
}

void RedBlackTree::deleteCase4(Node*& root, Node* node) {
    Node* sibling = findSibling(node);
    if (!sibling) return;

    // Явная проверка цвета родителя
    if (node->parent->color == RED &&
        sibling->color == BLACK &&
        (!sibling->left || sibling->left->color == BLACK) &&
        (!sibling->right || sibling->right->color == BLACK))
    {
        sibling->color = RED;
        node->parent->color = BLACK;
    } else {
        deleteCase5(root, node);
    }
}

void RedBlackTree::deleteCase5(Node*& root, Node* node) {
    Node* sibling = findSibling(node);

    if (sibling && sibling->color == BLACK) {

```

```

        if (node == node->parent->left && sibling->left &&
            sibling->left->color == RED &&
            (!sibling->right || sibling->right->color == BLACK)) {
            sibling->color = RED;
            sibling->left->color = BLACK;
            rightTurn(root, sibling);
        } else if (node == node->parent->right && sibling->right &&
            sibling->right->color == RED &&
            (!sibling->left || sibling->left->color == BLACK)) {
            sibling->color = RED;
            sibling->right->color = BLACK;
            leftTurn(root, sibling);
        }
    }
    deleteCase6(root, node);
}

void RedBlackTree::deleteCase6(Node*& root, Node* node) {
    Node* sibling = findSibling(node);

    if (!sibling) return;

    sibling->color = node->parent->color;
    node->parent->color = BLACK;

    if (node == node->parent->left) {
        if (sibling->right) {
            sibling->right->color = BLACK;
        }
        leftTurn(root, node->parent);
        if (node->parent == root) root = sibling;
    } else {
        if (sibling->left) {
            sibling->left->color = BLACK;
        }
        rightTurn(root, node->parent);
    }

    if (!sibling->parent) root = sibling;
}

void RedBlackTree::replaceNode(Node*& root, Node* node) {
    Node* replacement = node->left ? node->left : node->right;

    if (replacement) {
        replacement->parent = node->parent;

        if (!node->parent) {
            root = replacement;
        } else if (node == node->parent->left) {
            node->parent->left = replacement;
        } else {

```

```

        node->parent->right = replacement;
    }

    if (node->color == BLACK && replacement->color == BLACK) {
        deleteCase1(root, replacement);
    } else {
        replacement->color = BLACK;
    }

} else if (!node->parent) {
    root = nullptr;
} else {
    if (node->color == BLACK) {
        deleteCase1(root, node);
    }

    if (node == node->parent->left) {
        node->parent->left = nullptr;
    } else {
        node->parent->right = nullptr;
    }
}
}

void RedBlackTree::removeOne(Node*& root, Node* node) {
    if (node->left && node->right) {
        // Исправлено: ищем минимальный в правом поддереве
        Node* replacement = node->right;
        while (replacement->left)
            replacement = replacement->left;

        // Переносим ВСЕ данные, а не только значение
        Node tmp = *replacement;
        replaceNode(root, replacement);
        node->value = tmp.value;
        node->color = tmp.color; // Сохраняем исходный цвет
    } else {
        replaceNode(root, node);
    }
}

// Helper methods for printing
std::string RedBlackTree::formatNode(Node* node) {
    if (!node)
        return "    ";
    string val = to_string(node->value);
    return (node->color == RED ? "R" : "B") + val + string(3 - val.size(), ' ');
}

void RedBlackTree::printHelper(Node*** nodes, Node* current, size_t depth,
size_t index) {

```

```

nodes[depth][index] = current;
if (current->left) {
    printHelper(nodes, current->left, depth + 1, 2 * index);
}
if (current->right) {
    printHelper(nodes, current->right, depth + 1, 2 * index + 1);
}
}

// Helper method for calculating tree height
void RedBlackTree::max_height(Node* root, size_t& max, size_t depth) {
    if (!root) {
        return;
    }

    if (depth > max) {
        max = depth;
    }

    max_height(root->left, max, depth + 1);
    max_height(root->right, max, depth + 1);
}

```

IV.2. Анализ

Повороты

Левый и правый поворот работают за константное время.

Поиск

Любое красно-черное дерево с n узлами имеет высоту $h = \log(n)$, так как оно сбалансированное.

Обходы

Все обходы занимают $O(n)$, так как нужно пройти по всем элементам.

Вставка

Вставка в непустое дерево состоит из трех этапов:

1. Обычная вставка в дерево поиска: так как кчд является сбалансированным, то вставка занимает $O(\log n)$.
2. Окраска нового узла: происходит за константное время.
3. Восстановление свойств: так как восстановление свойств в худшем случае состоит из не более чем 2 поворотов, каждый из которых занимает $O(1)$. В случае, если требуется перекрасить другие узлы дерева, то перекраска будет идти до корня (его цвет в любом случае черный). Высота дерева составляет $\log(n)$, соответственно в худшем случае перекраска будет занимать $O(\log(n))$.

Так как операция вставки и восстановления выполняются последовательно, то по правилу суммы общая временная сложность вставки нового элемента в дерево будет занимать $O(\log(n))$

Удаление

- Нахождение узла для удаления занимает $O(\log n)$, так как дерево сбалансированное.
- Затем производится восстановление свойств дерева, которое в худшем случае требует 3 поворота, которые занимают $O(1)$.
- Перекраска для сохранения свойств выполняется аналогично случаю со вставкой.

Общая временная сложность удаления будет занимать $O(\log n)$.

V. AVL-дерево**V.1. Реализация**

```
#pragma once
#include <iostream>
#include <string>
struct AVLNode {
    int value;
    AVLNode *left;
    AVLNode *right;
    int height;

    AVLNode(int k) : value(k), left(nullptr), right(nullptr), height(1) {
    }
};

struct AVLTree {
    AVLTree() : root(nullptr) {
    }

    void insert(int key);
    void remove(int key);
    void print();

    AVLNode* find(int value);

private:
    AVLNode *root;

    AVLNode* find_recursive(AVLNode* node, int key);
    int height(AVLNode *node);
    int balance_factor(AVLNode *node);
    AVLNode *right_rotate(AVLNode *y);
    AVLNode *left_rotate(AVLNode *x);
    AVLNode *insert_helper(AVLNode *node, int key);
    AVLNode *min_value_node(AVLNode *node);
    AVLNode *delete_node(AVLNode *node, int key);
    void print_helper(AVLNode *root, std::string indent, bool last);
};
#include "avl_tree.h"
```



```
int AVLTree::height(AVLNode *node) {
    if (node == nullptr)
        return 0;
    return node->height;
}

int AVLTree::balance_factor(AVLNode *node) {
    if (node == nullptr)
        return 0;
    return height(node->left) - height(node->right);
}

AVLNode *AVLTree::right_rotate(AVLNode *node) {
    AVLNode *leftNode = node->left;
    AVLNode *T2 = leftNode->right;

    leftNode->right = node;
    node->left = T2;

    node->height = std::max(height(node->left), height(node->right)) + 1;
    leftNode->height =
        std::max(height(leftNode->left), height(leftNode->right)) + 1;

    return leftNode;
}

AVLNode *AVLTree::left_rotate(AVLNode *node) {
    AVLNode *rightChild = node->right;
    AVLNode *T2 = rightChild->left;

    rightChild->left = node;
    node->right = T2;

    node->height = std::max(height(node->left), height(node->right)) + 1;
    rightChild->height =
        std::max(height(rightChild->left), height(rightChild->right)) + 1;

    return rightChild;
}

AVLNode *AVLTree::insert_helper(AVLNode *node, int key) {
    if (node == nullptr) {
        return new AVLNode(key); // O(1)
    }
    if (key < node->value) {
        node->left = insert_helper(node->left, key);
    }
    else if (key > node->value) {
        node->right = insert_helper(node->right, key);
    }
    else {
        return node;
    }
}
```

```

    }

    node->height = 1 + std::max(height(node->left), height(node->right));

    int balance = balance_factor(node);

    // Левый левый случай
    if (balance > 1 && key < node->left->value) {
        return right_rotate(node);
    }

    // Левый правый случай
    if (balance < -1 && key > node->right->value) {
        return left_rotate(node);
    }

    // Правый правый случай
    if (balance > 1 && key > node->left->value) {
        node->left = left_rotate(node->left);
        return right_rotate(node);
    }

    // Правый левый случай
    if (balance < -1 && key < node->right->value) {
        node->right = right_rotate(node->right);
        return left_rotate(node);
    }
    return node;
}

void AVLTree::print_helper(AVLNode *root, std::string indent, bool last) {
    if (root != nullptr) {

        std::cout << indent;
        if (last) {
            std::cout << "R";
            indent += " ";
        }
        else {
            std::cout << "L";
            indent += " ";
        }
        std::cout << root->value << "(" << root->height << ")" << std::endl;
        print_helper(root->left, indent, false);
        print_helper(root->right, indent, true);
    }
}

void AVLTree::print() {
    if (root == nullptr) {
        std::cout << "У дерева нет корня" << std::endl;
    }
    else {

```

```

        print_helper(root, "", true);
    }
}

AVLNode* AVLTree::find(int value) {
    return find_recursive(root, value);
}

AVLNode* AVLTree::find_recursive(AVLNode* node, int value) {
    if (!node || node->value == value) return node;
    if (value < node->value) return find_recursive(node->left, value);
    return find_recursive(node->right, value);
}

AVLNode *AVLTree::min_value_node(AVLNode *node) {
    AVLNode *current = node;
    while (current->left != nullptr)
        current = current->left;
    return current;
}

AVLNode *AVLTree::delete_node(AVLNode *node, int key) {
    if (node == nullptr) {
        return node;
    }

    if (key < node->value) {
        node->left = delete_node(node->left, key);
    }
    else if (key > node->value) {
        node->right = delete_node(node->right, key);
    }
    else {
        if ((node->left == nullptr) || (node->right == nullptr)) {
            AVLNode *temp = node->left ? node->left : node->right;
            if (temp == nullptr) {
                temp = node;
                node = nullptr;
            }
            else {
                *node = *temp;
            }
            delete temp;
        }
        else {
            AVLNode *temp = min_value_node(node->right);
            node->value = temp->value;
            node->right = delete_node(node->right, temp->value);
        }
    }

    if (node == nullptr) {

```

```

        return node;
    }

    node->height = 1 + std::max(height(node->left), height(node->right));

    int balance = balance_factor(node);

    // Левый левый случай
    if (balance > 1 && balance_factor(node->left) >= 0) {
        return right_rotate(node);
    }

    // Левый правый случай
    if (balance > 1 && balance_factor(node->left) < 0) {
        node->left = left_rotate(node->left);
        return right_rotate(node);
    }

    // Правый правый случай
    if (balance < -1 && balance_factor(node->right) <= 0)
        return left_rotate(node);

    // Правый левый случай
    if (balance < -1 && balance_factor(node->right) > 0) {
        node->right = right_rotate(node->right);
        return left_rotate(node);
    }
    return node;
}

void AVLTree::insert(int key) {
    root = insert_helper(root, key);
}

void AVLTree::remove(int key) {
    root = delete_node(root, key);
}

```

V.2. Анализ

VI. Анализ сложности

N — количество элементов в дереве

Случаи АВЛ дерева

AVL дерево является сбалансированным деревом, что означает, что для него всегда выполняется лучший случай.

Высота узла

Функция возвращает высоту узла в дереве из поля `height`, что занимает $O(1)$.

Фактор баланса

Если узел равен `null`, то возвращается 0. Если узел нет, то вычислит разницу между высотами дерева левого и правого ребенка за $O(1) + O(1) = O(1)$

Повороты и балансировка

Высота всегда будет $\log n$, так как дерево балансируется поворотами.

Любой поворот перопределяет указатели и высоты узлов в дереве. Все операции в повороте займут $O(1)$. В худшем случае требуется не более двух поворотов для восстановления баланса за $O(1)$ дерева. Временная сложность составит $O(\log n)$

Вставка

- Сначала ищем узел за $O(\log N)$, так как дерево балансируемое.
- После этого идет балансировка — $O(O(\log N))$.

Общая сложность вставки $O(\log N) + O(\log N) = O(\log N)$

Удаление

- Сначала ищем узел за $O(\log N)$, так как дерево балансируемое.
- Удаление узла, занимает от $O(1)$ (если нет детей у узла) до $O(\log N)$ (при наличии двух детей), как в бинарном дереве, так как надо заменить на минимальный из его правого поддерева. Поиск минимума займет $O(\log N)$
- После этого идет балансировка — $O(O(\log N))$.

Обходы

Все обходы занимают $O(n)$, так как нужно пройти по всем элементам.

Затраты по памяти

Расход памяти за дерево составляет $O(N)$, тк в максимальное количество элементов в памяти равно количеству узлов в дереве.

VII. Анализ префикс, Z и КМП

VII.1. Префикс-функция

Реализация

```
std::vector<int> prefix_function(std::string str) {
    // Массив под результат префикс функции
    std::vector<int> prefixArr(str.length());
    prefixArr[0] = 0;

    for (size_t i = 1; i < str.length(); i++) {
        int k = prefixArr[i - 1];

        while (k > 0 && str[i] != str[k]) {
            k = prefixArr[k - 1];
        }

        if (str[i] == str[k]) {
```

```

        k++;
    }
    prefixArr[i] = k;
}
return prefixArr;
}

```

Анализ

1. Инициализация массива за $O(n)$
2. Внешний цикл работает за $O(n)$
 1. Индексация и присвоение текущего префикса за $O(1)$
 2. Внутренний цикл while в худшем случае выполняется не более n раз за все итерации внешнего цикла. Значит сложность $O(n)$, при условии, что правило произведения тут не действует.

Общая временная сложность будет $O(n) + O(n) = O(n)$

VII.2. Z-функция

Реализация

```

std::vector<int> z_function(std::string str) {
    // Массив под результат z-функции
    std::vector<int> zf(str.length());

    int left = 0, right = 0;

    for (size_t i = 1; i < str.length(); i++) {
        zf[i] = std::max(0, std::min(right - static_cast<int>(i),
zf[static_cast<int>(i) - left]));

        while (i + zf[i] < str.length() && str[zf[i]] == str[i + zf[i]]) {
            zf[i]++;
        }
        if (i + zf[i] > static_cast<size_t>(right)) {
            left = i;
            right = i + zf[i];
        }
    }
    return zf;
}

int z_search_substring(std::string haystack, std::string needle) {
    // Массив z-функции
    std::vector<int> zf = z_function(needle + '#' + haystack);

    // Проходимся по массиву
    for (size_t i = needle.length(); i < zf.size(); i++) {
        // Если z-функция равна длине needle, то нашли needle
        if (static_cast<size_t>(zf[i]) == needle.length()) {
            // Выводим найденный индекс
            return i - needle.length() - 1;
        }
    }
}

```

```

    }
}
return -1;
}
int substring_search_prefix(std::string haystack, std::string needle) {
    // Массив z-функции
    std::vector<int> prefixArr = prefix_function(needle + '#' + haystack);

    // Проходимся по массиву
    for (size_t i = needle.length(); i < prefixArr.size(); i++) {
        // Если префикс функция равна длине needle, то нашли needle
        if (static_cast<size_t>(prefixArr[i]) == needle.length()) {
            // Выводим найденный индекс
            return i - (needle.length() * 2);
        }
    }
    return -1;
}
}

```

Анализ

В данной реализации каждый символ строки рассматривается не более двух раз: один раз при увеличении i и один раз внутри цикла while.

1. Инициализация массива за $O(n)$
2. Внешний цикл работает за $O(n)$
 1. Внутри цикла все сравнения выполняются за $O(1)$
 2. Когда выполняется сравнение пары символов, правая граница сдвигается, но не больше чем на n символов. за все итерации внешнего цикла.

Общая временная сложность будет $O(n) + O(n) = O(n)$

VII.3. КМП

Реализация

```

std::vector<int> kmp(std::string haystack, std::string needle) {
    int needle_length = needle.length();
    int haystack_length = haystack.length();
    std::vector<int> answer(haystack_length);

    // Значение префикс-функции
    std::vector<int> p = prefix_function(needle + "#" + haystack);

    int count = 0;
    for (size_t i = needle_length; i < p.size(); i++) {
        if (p[i] == needle_length) {
            answer[count++] = static_cast<int>(i) - 2 * needle_length + 1; //
+ если считать с 1
        }
    }
    return answer;
}

```

Анализ

n — длина строки, k — длина паттерна

- Инициализация массива за $O(n)$
- Префикс функция работает за $O(n + k)$
- Цикл работает за $O(n)$

Таким образом общая временная сложность будет: $O(n) + O(n + k) + O(n) = O(n + k)$

VIII. Анализ Бойера-Мура**VIII.1. Реализация**

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>
#include "strings.h"

#define ALPHABET_LEN 256

void pre_bm_bc(std::vector<int> &table, std::string pattern) {
    for (int i = 0; i < ALPHABET_LEN; i++) {
        table[i] = pattern.length();
    }
    for (size_t i = 0; i < pattern.length() - 1; i++) {
        table[pattern[i]] = pattern.length() - 1 - static_cast<int>(i);
    }
}

bool is_prefix(std::string word, int pos) {
    int suffixLen = word.length() - pos;

    for (int i = 0; i < suffixLen; i++) {
        if (word[i] != word[pos + i]) {
            return false;
        }
    }
    return true;
}

int suffix_length(std::string word, int pos) {
    int len = 0;
    for (len = 0;
         (word[pos - len] == word[word.length() - 1 - len]) && (len < pos);
         len++);
    return len;
}
```



```

void pre_bm_gc(std::vector<int> &table, std::string pattern) {
    int lastPrefixIndex = 1;
    // Этап 1: Находим границы совпадений
    for (size_t pos = pattern.length(); pos > 0; pos--) {
        if (is_prefix(pattern, pos)) {
            lastPrefixIndex = pos;
        }
        table[pos] = (pattern.length() - 1 - pos - 1) + lastPrefixIndex;
    }

    // Этап 2: Заполняем таблицу хороших суффиксов
    for (size_t pos = 0; pos < pattern.length() - 1; pos++) {
        int suffixLen = suffix_length(pattern, pos);
        if (pattern[pos - suffixLen] !=
            pattern[pattern.length() - 1 - suffixLen]) {
            table[pattern.length() - 1 - suffixLen] =
                pattern.length() - 1 - pos + suffixLen;
        }
    }
}

std::vector<int> bm(std::string haystack, std::string needle) {
    std::vector<int> badChar(ALPHABET_LEN);
    std::vector<int> goodSuf(needle.length());
    std::vector<int> result;
    size_t haystack_len = haystack.length();
    size_t needle_len = needle.length();

    if (needle_len == 0 || needle_len > haystack_len) {
        return result;
    }

    pre_bm_bc(badChar, needle);
    pre_bm_gc(goodSuf, needle);

    size_t i = needle_len - 1; // Позиция в тексте
    while (i < haystack_len) {
        int j = needle_len - 1; // Позиция в шаблоне
        size_t start_pos = i; // Запоминаем начальную позицию для сравнения

        // Сравниваем справа налево
        while (j >= 0 && haystack[i] == needle[j]) {
            --i;
            --j;
        }

        if (j < 0) {
            // Найдено совпадение
            result.push_back(i + 1);
            // Восстанавливаем позицию и делаем минимальный сдвиг
            i = start_pos + 1;
        } else {

```

```

// Сдвиг по максимальному из двух правил
size_t bad_char_shift = (haystack[i] < ALPHABET_LEN) ?
    std::max(1, j - badChar[haystack[i]]) : j + 1;
size_t good_suffix_shift = (j < needle_len - 1) ?
    goodSuf[j + 1] : 1;

    i = start_pos + std::max(bad_char_shift, good_suffix_shift);
}
}
return result;
}

```

VIII.2. Анализ

n — длина строки/текста, в котором мы ищем шаблон

m — длина шаблона (искомой строки)

Построение таблиц

Построение таблиц происходит за $O(m)$ и для “Плохих символов” и “Хороших суффиксов”.

Поиск подстроки

В основном цикле количество сдвигов не может превышать $\frac{n}{m}$, причем каждый сдвиг как минимум на одну позицию.

При выполнении каждого сдвига берутся данные из таблиц “Плохих символов” и “Хороших суффиксов” и из них выбирается максимальное значение. В худшем случае определение каждого займет $O(n \cdot m)$.

Следовательно, в худшем случае алгоритм работает за:

$$O(m) + O(m) + O(n) \cdot O(m) = O(n \cdot m)$$

В лучшем случае алгоритм, применяя таблицы, будет выполнять крупные сдвиги. Это сокращает число сравнений и сдвигов. При этом общее количество сдвигов не превышает $\frac{n}{m}$:

$$O(m) + O(m) + O\left(\frac{n}{m}\right) \cdot O(m) = O(n)$$

В среднем случае алгоритм так же работает за $O(n)$